

Université de Pau et des Pays de l'Adour

Master Technologies de l'Internet – M1

Département d'Informatique

r/Place Clicker

Rapport de projet développement web

Réalisé par Missipsa BOUHADAD
Loïc Quessette
Michaël Darricau

Encadrante Lea Brunschwig

Année universitaire 2025–2026

Table des matières

1 Introduction **2**

2 Choix technologiques **2**
2.1 Côté serveur 2
2.2 Base de données 2
2.3 Côté client 2

3 Architecture **3**
3.1 Diagramme d'architecture 3

4 Partie données **4**
4.1 Schéma entité-association 4
4.2 Modèle relationnel 4
4.3 Traduction entités – tables en BDD 5

5 Diagrammes UML **5**
5.1 Diagramme de classe 5
5.2 Diagramme de cas d'utilisation 6

6 Lancer l'application **7**
6.1 Prérequis 7

7 Scénario d'utilisation **8**

8 Répartition du travail **8**

1 Introduction

Le projet consiste en une grille collaborative de 50x50 pixels accessible en temps réel par tous les utilisateurs. Chaque joueur authentifié peut poser des pixels sur la grille en dépensant des crédits, et optimiser sa récolte de crédits via un système de bonus. Une page de statistiques permet de suivre les performances de chaque joueur : nombre de pixels placés, couverture de la grille, pixel le plus ancien encore en place ainsi que le record d'âge d'un pixel.

2 Choix technologiques

2.1 Côté serveur

- **Apache Tomcat 10** : serveur choisi pour déployer l'application. Tomcat est léger, open source et conçu pour les applications Java web basées sur les Servlets et JSP.
- **Java Servlets / JSP + JSTL** : les Servlets jouent le rôle de contrôleur (gestion des requêtes HTTP), les JSP s'occupent de l'affichage, et JSTL permet d'éviter d'écrire du code Java brut dans les pages JSP.
- **JPA avec Hibernate** : utilisé pour ne pas écrire du SQL à la main pour chaque requête, en travaillant directement avec des objets Java tout en gardant la couche d'accès aux données bien séparée du reste de l'application.

2.2 Base de données

- **MySQL** : base de données relationnelle qui fonctionne avec JPA via Hibernate, très adaptée à notre projet qui nécessite de stocker des joueurs, des pixels, un historique et des bonus.

2.3 Côté client

- **Tailwind CSS** : utilisé pour styliser les pages JSP via des classes utilitaires sans gérer de fichiers CSS séparés.
- **JavaScript** : gère côté client la connexion WebSocket avec le serveur, permettant de recevoir et d'envoyer des événements en temps réel (pose d'un pixel, mise à jour de la grille), ainsi que le système de clicker et la sauvegarde automatique des crédits.

3 Architecture

3.1 Diagramme d'architecture

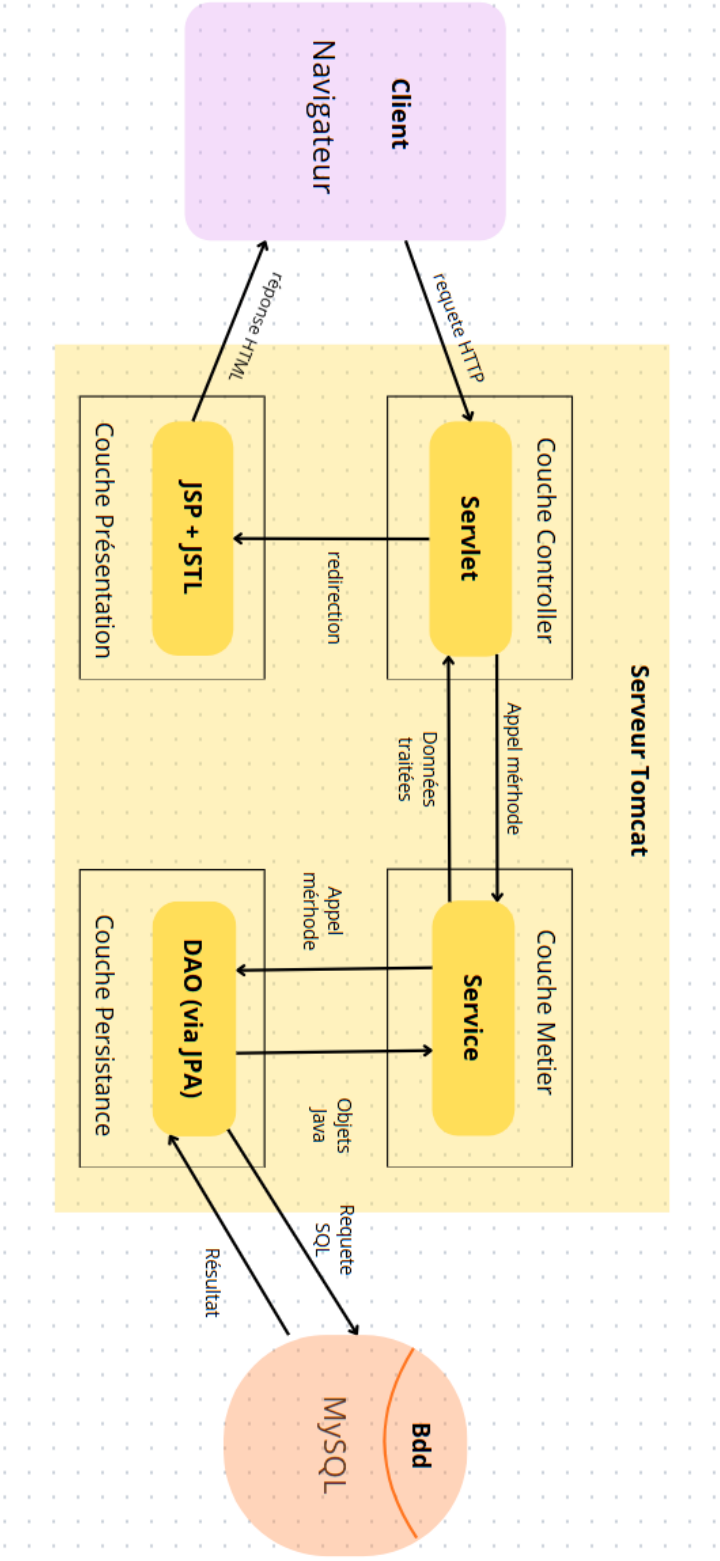


FIGURE 1 – Diagramme d'architecture

4 Partie données

4.1 Schéma entité-association

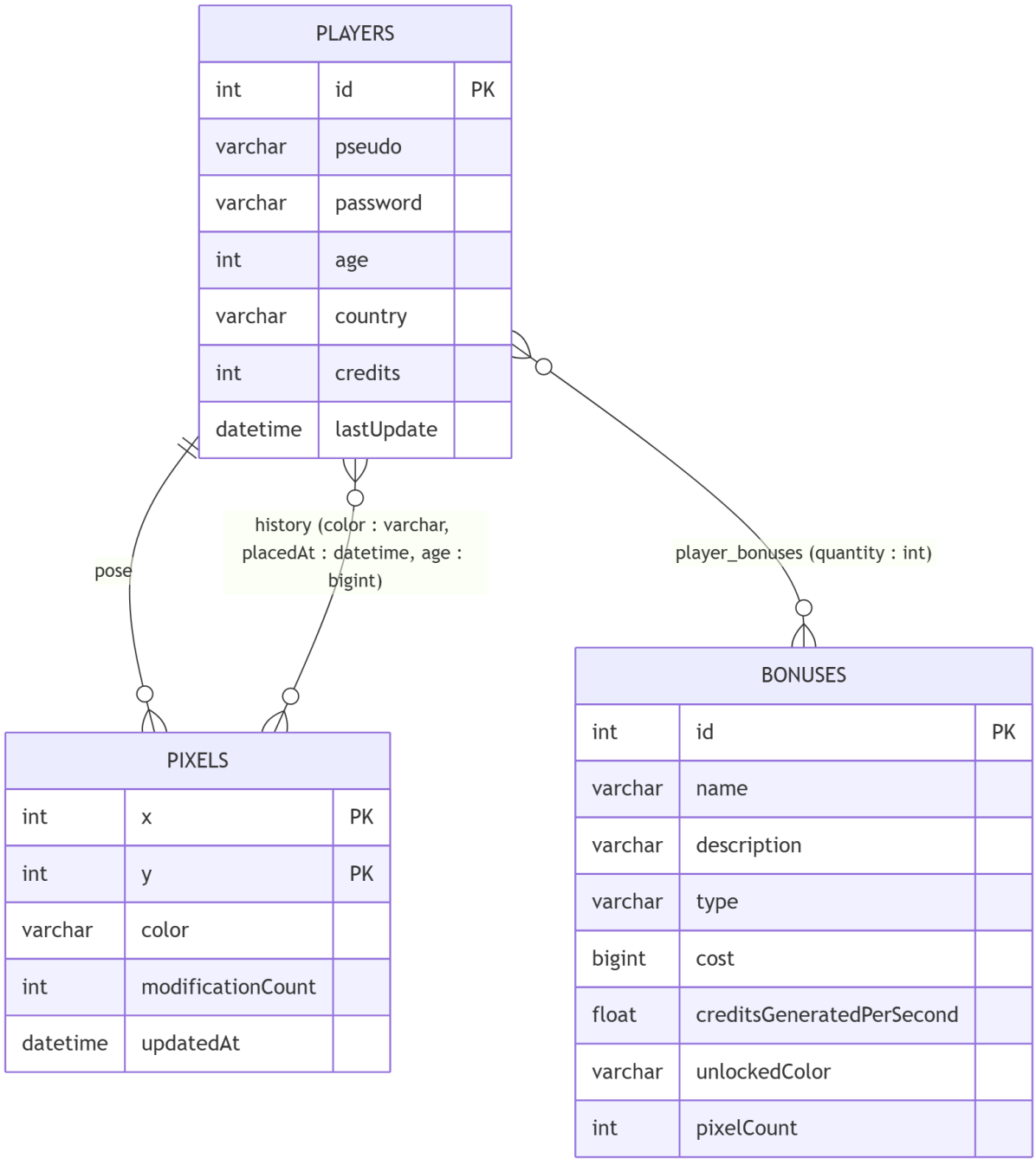


FIGURE 2 – Schéma entité association

4.2 Modèle relationnel

- **players**(<u>id</u>, pseudo, password, age, country, credits, lastUpdate)
- **pixels**(<u>x, y</u>, color, modificationCount, updatedAt, #playerId)
- **history**(<u>id</u>, x, y, color, placedAt, age, #playerId)

— **bonuses**(id, name, description, type, cost, creditsGeneratedPerSecond, unlockedColor, pixelCount)
— **player_bonuses**(id, #playerId, #bonusId, quantity)

4.3 Traduction entités – tables en BDD

Chaque table du MLD devient une entité Java :

Player et Bonus sont des entités classiques, chaque attribut du MLD devient un champ avec `@Column`.

Pixel a une clé primaire composite (x, y) on a donc créé une classe séparée `PixelId` avec `@Embeddable` pour gérer ça, et on l'utilise avec `@EmbeddedId` dans `Pixel`. Il fallait aussi implémenter `equals()` et `hashCode()` dans `PixelId` sinon JPA ne sait pas comparer deux pixels.

History et PlayerBonus correspondent aux associations n-aires du MCD. Elles sont traduites en entitées à part avec leur propre id (auto incrémenté). Les références vers `Player`, `Bonus` et `Pixel` sont ensuite gérées avec des `@ManyToOne`.

A noter que dans `Bonus` les champs `currentPrice` et `userQuantityOwned` sont annotés `@Transient` car ils ne sont pas stockés en base, ils sont juste calculés à la volée selon le joueur connecté.

5 Diagrammes UML

5.1 Diagramme de classe

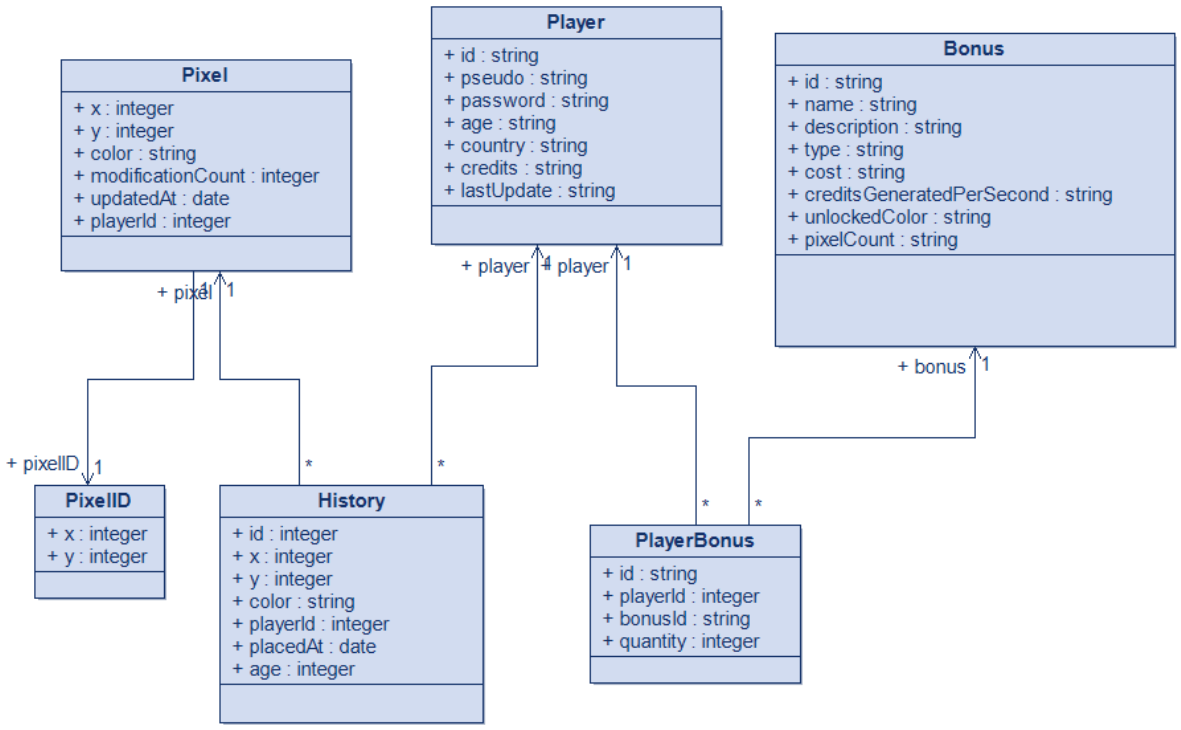


FIGURE 3 – Diagramme de classe

5.2 Diagramme de cas d'utilisation

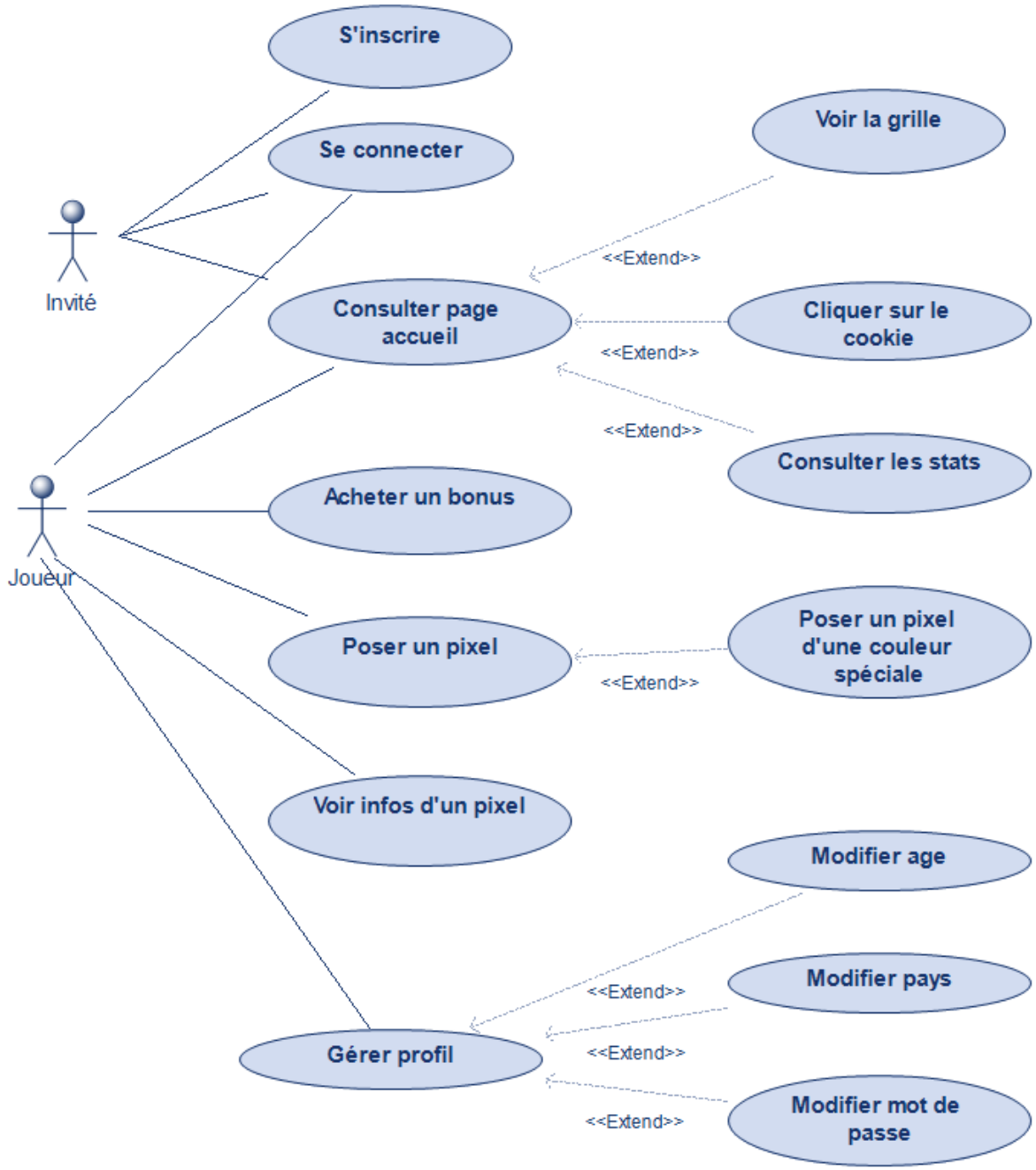


FIGURE 4 – Diagramme de cas d'utilisation

6 Lancer l'application

6.1 Prérequis

— Java 17+
— Maven
— Apache Tomcat 10+
— MySQL

1. **Cloner le projet**
   `git clone https://git.univ-pau.fr/mdarricau004/cookie.git`

2. **Créer la base de données**
   Ouvrir MySQL Workbench et créer une base de données nommée `rplace_clicker_db`.

3. **Configurer les identifiants de connexion**
   Dans le fichier `JpaInitializer.java`, remplacez `dbUser` par votre nom d'utilisateur MySQL (par défaut `root`) et `dbPassword` par votre mot de passe MySQL (par défaut `root`).

4. **Compiler le projet**
   Ouvrir un terminal dans le dossier du projet et lancer :
   `mvn package`

5. **Déployer sur Tomcat**

   Sous Windows :
   Supprimer l'ancien dossier ROOT s'il existe :

   ```
   Remove-Item -Recurse -Force votre_chemin_tomcat\webapps\ROOT
   ```

   Puis copier le WAR généré vers Tomcat :

   ```
   copy target\rplace-clicker-1.0-SNAPSHOT.war votre_chemin_tomcat\
   webapps\ROOT.war
   ```

   Sous Mac :
   Supprimer l'ancien dossier ROOT s'il existe :

   ```
   rm -rf votre_chemin_tomcat/webapps/ROOT
   ```

   Puis copier le WAR généré vers Tomcat :

   ```
   cp target/rplace-clicker-1.0-SNAPSHOT.war votre_chemin_tomcat/
   webapps/ROOT.war
   ```

6. **Lancer Tomcat**

   Sous Windows :
   Aller dans le dossier `bin` de Tomcat et lancer :

   ```
   startup.bat
   ```

Sous Mac :
Aller dans le dossier `bin` de Tomcat et lancer :

```
startup.sh
```

Ou avec Homebrew :

```
brew services start tomcat
```

7. **Ouvrir le navigateur**
   Accéder à :
   `http://localhost:8080`

8. **Vérification**
   Sur MySQL Workbench, vérifier que les tables ont été créées automatiquement dans `rplace_clicker_db`.

7 Scénario d'utilisation

1. **Créer un compte** : cliquer sur "S'inscrire", renseigner pseudo, mot de passe, âge et pays.
2. **Se connecter** : saisir pseudo et mot de passe.
3. **Miner des crédits** : cliquer sur le bouton de minage pour gagner des crédits rapidement.
4. **Acheter des bonus** : dans la boutique, acheter des générateurs pour automatiser la récolte.
5. **Poser un pixel** : ouvrir la grille, choisir une couleur dans la palette, cliquer sur une case.
6. **Consulter les statistiques** : accéder à la page Statistiques pour voir le classement et les informations des joueurs.

8 Répartition du travail

Membre	Travail fait
Missipsa Bouhadad	Connexion, Inscription, Affichage de la grille, Ajouter une palette de couleurs, Poser un pixel, Afficher les statistiques des joueurs
Loïc Quessette	Modification du profil, Boutique de bonus passifs, Achat de couleurs bonus, Affichage des bonus débloqués
Michaël Darricau	Auto-save et save, Clicker, Payment pixel, Contabilisation des bonus dans les credits possedés, Affichage de la grille en petit format, Section mes statistiques